



GUÍA TÉCNICA

PYTHON · IA SIMBÓLICA

Sistemas Expertos con *experta* en Python

Cómo construir inteligencia artificial basada en reglas usando la librería *experta*, desde los conceptos fundamentales hasta la implementación práctica de un motor de inferencia.



CAPÍTULO 1

¿Qué es un Sistema Experto?

Un **sistema experto** es un programa de IA simbólica que combina un conjunto de *hechos* (información) con **reglas de inferencia** estilo "Si-Entonces" para tomar decisiones o resolver problemas especializados. Son la forma más exitosa de IA simbólica.

Por ejemplo, un sistema experto puede diagnosticar averías siguiendo reglas basadas en el conocimiento de un experto humano, replicando su razonamiento de forma automatizada.

Hechos

Datos e información del dominio

Reglas

Lógica "Si-Entonces" del experto

Inferencia

Motor que combina hechos y reglas

La librería *experta*

La librería **experta** (un fork de PyKnow) es una biblioteca de Python inspirada en **CLIPS** que permite crear sistemas expertos declarativos. Con ella definimos hechos y reglas en clases de Python, y el motor de inferencia aplica automáticamente esas reglas a los hechos conocidos.

Fact (Hecho)

Objeto basado en `dict` que almacena datos etiquetados.

@Rule (Regla)

LHS (antecedente): patrones que deben coincidir. **RHS** (consecuente): acciones a ejecutar.

KnowledgeEngine

Motor de inferencia. Con `reset()` y `run()` aplica las reglas a los hechos disponibles.



Clave: El motor separa el conocimiento (reglas) de los datos (hechos) y los combina para inferir nuevas conclusiones automáticamente.

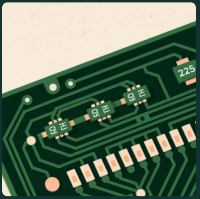
Hechos (*Facts*)

Los **hechos** son la unidad básica de información en *experta*. Internamente, `Fact` es una subclase de `dict`, por lo que permite acceder a valores con claves.

```
from experta import *  
  
# Crear un hecho  
f = Fact(animal='gato', color='gris')  
# Acceder a valores  
f['animal'] # → 'gato'
```

Cada hecho describe una pieza de conocimiento del dominio. El motor los almacena en su memoria de trabajo y los compara con los patrones de las reglas para determinar qué acciones ejecutar.

Reglas (*Rules*) y Motor de Inferencia



Reglas @Rule

Se definen anotando métodos con `@Rule(...)`. La **LHS** contiene patrones con operadores lógicos (`AND`, `OR`, `NOT`). La **RHS** ejecuta acciones como imprimir o declarar nuevos hechos.



KnowledgeEngine

Clase donde se definen todas las reglas. `reset()` inicializa la memoria (incluyendo `InitialFact` y hechos de `@DefFacts`). `declare()` añade hechos y `run()` ejecuta el ciclo de inferencia.

Ejemplo de regla

```
@Rule(Fact(color='green'))
def regla_verde(self):
    print("Color detectado: verde")
```

Instalación y Primeros Pasos

01

Instalar la librería

`pip install experta` desde PyPI.

03

Crear reglas en KnowledgeEngine

Heredar de `KnowledgeEngine` y anotar métodos con `@Rule` definiendo patrones y acciones.

02

Definir hechos

Crear clases que hereden de `Fact` para estructurar la información del dominio.

04

Ejecutar el motor

Instanciar → `reset()` → `declare()` hechos → `run()` para iniciar la inferencia.

```
from experta import *

class LuzSemaforo(Fact):
    """Hecho que describe el estado de un semáforo."""
    pass
```

Robot que Decide en un Semáforo

Veamos un ejemplo completo: un robot decide si puede cruzar según la luz de un semáforo. Creamos el hecho `LuzSemaforo` y el motor `CruceRobot` con tres reglas:

```
class CruceRobot(KnowledgeEngine):  
    @Rule(LuzSemaforo(color='green'))  
    def luz_verde(self):  
        print("Camina")  
  
    @Rule(LuzSemaforo(color='red'))  
    def luz_roja(self):  
        print("No cruces")  
  
    @Rule(AS.luz << LuzSemaforo(  
        color=L('yellow') | L('blinking-yellow')))  
    def precaucion(self, luz):  
        print(f"Precaución: la luz es {luz['color']}")
```



Ejecución y Explicación del Ejemplo

Código de ejecución

```
engine = CruceRobot()
engine.reset()
engine.declare(
    LuzSemaforo(color='green')
)
engine.run()
# Salida: "Camina"
```

¿Qué ocurre?

- `luz_verde` se dispara con color verde → imprime "Camina"
- `luz_roja` imprime "No cruces" para rojo
- `precaucion` usa **OR**: si color es 'yellow' o 'blinking-yellow', se activa
- `AS.luz <<` asigna el hecho a la variable `luz` para acceder a su contenido en el RHS

El motor busca qué reglas se cumplen con los hechos existentes y ejecuta sus acciones automáticamente. Este es el patrón típico en *experta*: declarar hechos, definir reglas con condiciones claras, y dejar que el motor gestione la inferencia.

Lógica de Inferencia: Encadenamiento hacia Adelante

experta utiliza **encadenamiento hacia adelante**, similar al de CLIPS. En cada ciclo de ejecución, el motor realiza los siguientes pasos de forma automática:



El flujo del programa no está codificado paso a paso: el desarrollador solo define hechos y reglas, y *experta* maneja el proceso de inferencia automáticamente.



✧ RESUMEN

Conclusiones y Puntos Clave

IA Simbólica

Los sistemas expertos son la forma más exitosa de IA simbólica: combinan hechos + reglas + inferencia.

Librería *experta*

Fork de PyKnow inspirado en CLIPS. Permite definir hechos (Fact), reglas (@Rule) y motores (KnowledgeEngine) en Python puro.

Patrón de uso

Instanciar → `reset()` → `declare()` → `run()`. El motor separa conocimiento de datos y razona automáticamente.



Referencias: Documentación oficial de *experta*; definiciones de sistemas expertos y motores de inferencia.